

TECNOLOGÍAS DE LA INFORMACIÓN EN LÍNEA

DESARROLLO DE APLICACIONES WEB

3 créditos

**Profesor:**

Ing. Manuel Bravo, Mg.

Ing. Tatiana Cobeña Macías, Mg.

Ing. Edison Solorzano, MG.

Asignatura	Semestre
DESARROLLO DE APLICACIONES WEB	Sexto

Tutorías: El profesor asignado, se publicará en el entorno virtual de aprendizaje online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección **CAFETERÍA VIRTUAL**.

PERÍODO ACADÉMICO:

MAYO 2023 / SEPTIEMBRE 2023

Contenido

Unidad IV: Sistema de Control de versiones.....	2
Tema 1: Sistema de control de versiones.....	2
Tema 2: DevOPS.....	6

Resultado de aprendizaje de la asignatura

Este curso provee a los estudiantes el conocimiento y la experiencia práctica para diseñar e implementar aplicaciones web cumpliendo con los estándares actuales y las buenas prácticas de programación que faciliten su mantenibilidad, escalabilidad y adaptabilidad.

Ilustraciones gráficas



Sabías que. - La presente imagen dentro del manual mostrara información interesante y novedosa de la asignatura.



Recuerde que. - La presente imagen dentro del manual permite recordar información que es relevante y que vas necesitar en tu vida profesional.



Comprueba tu aprendizaje. - Es un cuestionario de un conjunto de preguntas que se confecciona para obtener información con algún objetivo en concreto. Por cada tema de la unidad se tendrá un cuestionario que el estudiante debe resolver entre preguntas teóricas y prácticas.



Videos. - Para complementar contenidos de la unidad dentro del manual se tiene videos que permitirán al estudiante revisar y explorar conocimientos auditivos y visuales.



Curiosidades. - La presente imagen en el manual mostrara información que debes conocer de la asignatura.



Datos útiles. - La presente imagen en el manual mostrara información que deberás tomar en cuenta en otras unidades de la asignatura o de otras asignaturas en semestres superiores.



DESARROLLO DE APLICACIONES WEB



Unidad IV: Sistema de Control de versiones.

Resultado de aprendizaje de la unidad: Aplicar un sistema de control de versiones en la construcción de la aplicación web para el respectivo mantenimiento del software y control de cambios.



Tema 1: Sistema de control de versiones.



Un sistema de control de versiones o SVC (System Version Control) es una herramienta que registra los cambios realizados sobre un archivo o conjunto de archivos de un proyecto a lo largo del tiempo, de modo que puedas recuperar la versión de esos archivos en un estado anterior en el tiempo.

El sistema de control de versiones es de gran utilidad para entornos de desarrollo colaborativo donde varios diseñadores, maquetadores y/o programadores trabajan sobre un mismo proyecto. En todo momento tenemos los cambios que se realizan sobre cada uno de los ficheros por los diferentes programadores y podemos recuperar un fichero en un estado anterior.

En este capítulo nos vamos a centrar en las herramientas Git y Subversión, pero existen muchas otras como CVS, Mercurial, Bazaar, SourceSafe, etc.

CARACTERÍSTICAS DE UN SISTEMA DE CONTROL DE VERSIONES

Es un sistema que registra los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo, de modo que se pueda recuperar versiones específicas de los mismos en un determinado momento. Un SVC posee tres capacidades importantes:

- **Reversibilidad:** retornar a un estado anterior del proyecto en caso de fallos.
- **Concurrencia:** Muchas personas modificando el mismo código o documento.
- **Anotación:** Adjuntar información relevante de los cambios realizados.

Un sistema de control de versiones debe proporcionar:

- Mecanismo de almacenamiento de los elementos que deba gestionar (ej. archivos de texto, imágenes, documentación).
- Posibilidad de realizar cambios sobre los elementos almacenados (ej. Modificaciones parciales, añadir, borrar, renombrar o mover elementos).
- Registro histórico de las acciones realizadas con cada elemento o conjunto de elementos (normalmente pudiendo volver o extraer un estado anterior del producto).

CONCEPTOS BÁSICOS EN LOS SISTEMAS DE CONTROL DE VERSIONES

- **Repositorio:** lugar en el que se almacenan los datos actualizados e históricos de cambios (sistema de archivos en un disco duro, un banco de datos, entre otros).
- **Revisión:** Versión determinada de la información que se gestiona.
- **Tags:** Permiten identificar de forma fácil revisiones importantes en el proyecto.
- **Módulo:** Conjunto de directorios y/o archivos dentro del repositorio que pertenecen a un proyecto común.

- **Branch:** Es una copia del proyecto aislada, de forma que los cambios realizados no afecten al resto del proyecto y vice versa, excepto cuando los cambios sean "unidos" de un lado al otro.
- **Baseline:** Una revisión aprobada de un documento o fichero fuente, a partir del cual se pueden realizar cambios subsiguientes.
- **Checkout:** crea una copia de trabajo local desde el repositorio.
- **Push:** La acción que traslada los contenidos de la copia local del repositorio de un programador a la copia maestra del mismo.
- **Update/Pull/Fetch+Merge/Rebase:** Acción empleada para actualizar nuestra copia local del repositorio a partir de la copia maestra del mismo, además de actualizar la copia de trabajo con el contenido actual del repositorio local.
- **Conflicto:** Situación que surge cuando dos desarrolladores hacen un commit con cambios en la misma región del mismo fichero. El scv lo detecta, pero es el programador el que debe corregirlo.
- **Etiquetar:** Poner un nombre común a todos los ficheros bajo control de versiones de un proyecto en un instante determinado.
- **Rama:** Representa una línea de evolución de nuestro software. Podemos tener todas las que queramos, pero normalmente una es especial: trunk, master/main, entre otros.

TIPOS DE SCV

Por la forma de almacenar los contenidos:

Centralizados: Hay un único repositorio compartido por todos los desarrolladores. Sólo un responsable o grupo de responsables pueden llevar a cabo tareas administrativas en este repositorio. Es el caso de CVS y Subversion.

Distribuidos: Cada usuario tiene su propio repositorio. Los repositorios de todos los desarrolladores del proyecto pueden intercambiar información entre ellos. Es habitual disponer de un repositorio central que sirve como sincronización entre todos los repositorios de los desarrolladores. Es el caso de git, mercurial, bazaar, monotone, entre otros.

Por la forma de modificar los contenidos de la copia local:

Colaborativos: Varios desarrolladores pueden estar modificando el mismo fichero simultáneamente. Pueden aparecer conflictos por modificar varios desarrolladores la misma parte del archivo. Es el caso de git o mercurial.

Exclusivos: Sólo un desarrollador puede estar modificando un archivo a la vez, el resto sólo pueden leerlo. Una vez se han pasado los cambios al repositorio entonces ya lo puede modificar otro desarrollador, pero nuevamente sólo uno. Es el caso de RCS o de SourceSafe.

Hoy en día disponemos de muchos y diversos scm, tales como:

- bazaar que es el sucesor de arch.
- CVS
- darcs
- git
- mercurial
- monotone
- subversión
- plasticcm, breve introducción en la Wikipedia.
- perforce, breve introducción en la Wikipedia.
- bitkeeper.

GIT LAB



GitLab nació como un sistema de alojamiento de repositorios Git, es decir, un hosting para proyectos gestionados por el sistema de versiones Git. Sin embargo, alrededor de esta herramienta han surgido muchas otras herramientas muy interesantes para programadores y equipos de desarrollo, que envuelven todo el flujo del desarrollo y el despliegue de aplicaciones, test.

La principal diferencia entre GitLab y sus competidores es que GitLab se ofrece como un software libre que puedes descargar e instalar en cualquier servidor. Esta posibilidad permite usar GitLab para una empresa, profesional u organización en sus propios servidores, sin ningún coste adicional.

FUNCIONALIDADES DE GIT LAB

En GitLab podemos gestionar principalmente proyectos, Grupos y Snippets. Los proyectos son los protagonistas del sistema, básicamente repositorios de software gestionados por GitLab y todo el ecosistema GitLab. Los grupos son básicamente empresas y usuarios. Los snippets por su parte son como pedazos de código que puedes dejar para hacer cualquier cosa.

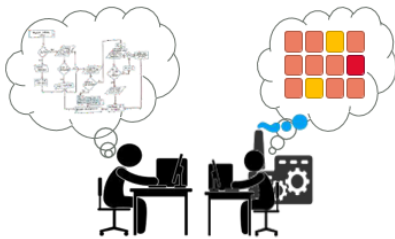
- SESIÓN DE ENTRENAMIENTO BÁSICO CON GIT.

Tema 2: DevOPS



Es una filosofía, un enfoque, no existe ningún estándar, metodología o herramienta como tal.

Pero sin embargo, cualquier iniciativa DevOps tiene como características comunes:



Acercar la **interacción** entre Desarrollo y Operaciones: “Funcionalidades vs. Tickets”

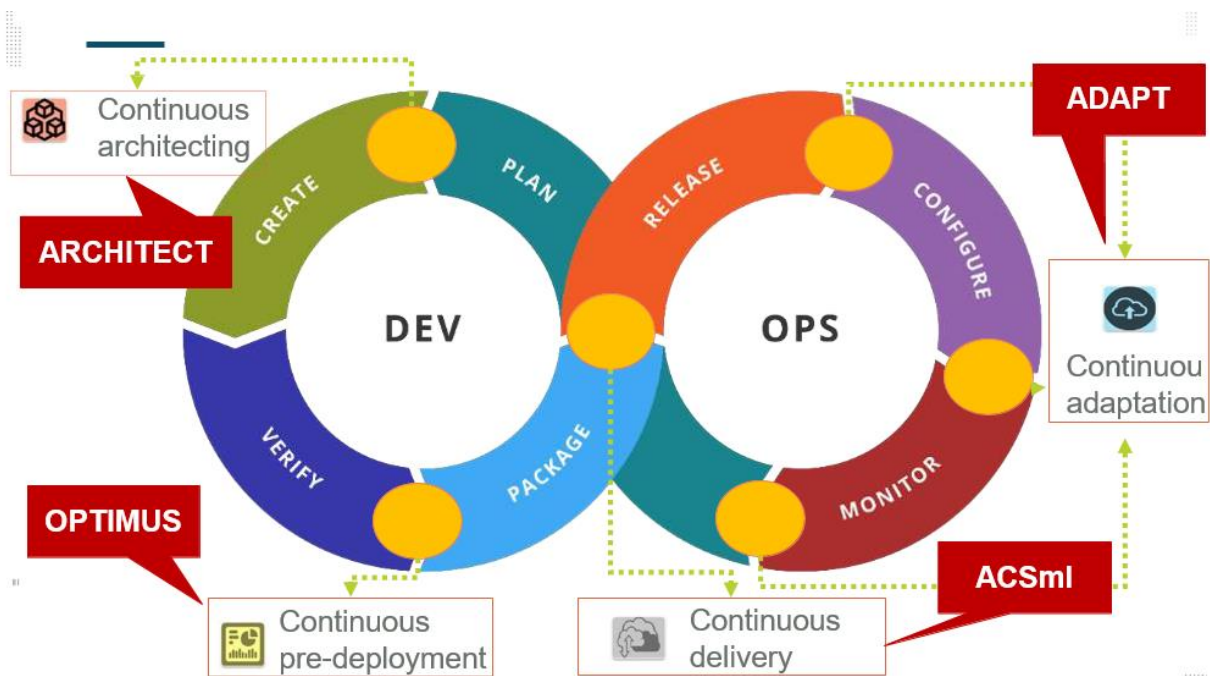


Entrega continua

(Continuous Delivery pipeline)

- Integración continua (CI)
- Calidad continua (CQ)
- Despliegue continuo (CD)

OTROS ENFOQUES DEL DEVOPS



Al final, cada empresa tiene que determinar que DevOps es el adecuado para su producto o servicio, pero siempre:

- Con un plan y objetivos claros.
- Alineado con sus necesidades de negocio.
- La formación y conocimiento del equipo
- Las características de su aplicación.

- Sus puntos críticos de mejora

PORQUE SURGIÓ DEVOPS

- La barrera entre el desarrollo y la operación cada vez es mayor.
- Los desarrollos son cada vez más complejos.
- El número de pasos para poner algo en producción crece.
- La frecuencia de despliegues es cada vez mayor.
- Tardar en recuperarse de un fallo supone perder un cliente.

BENEFICIOS DE DEVOPS



BARRERAS DE DEVOPS





Videos que permitirán complementar lo aprendido durante las sesiones.

SCV → <https://www.youtube.com/watch?v=zTwguRLU5DA>

DevOps → <https://www.youtube.com/watch?v=p-bOnV8FRMQ>

Curso de git desde cero → <https://www.youtube.com/watch?v=HiXLkL42tMU&t=40s>

Referencias Bibliográficas

Corbí, A. (2020). Sistemas de Control de Versiones de última generación (DCA). Recuperado desde: <https://www.dlsi.ua.es/asignaturas/dca/teoria/t10.pdf>

Miranda, J. (2016). Sistema de control de versiones. Recuperado desde: <https://www.profesordeinformatica.com/descargas/controldeversiones.pdf>

Bucheli, L. (2021). Presentación sobre DevOps.